TD 04: Typage

1 Noyau de OCaml

On rappelle le langage utilisé en CM, auquel on ajoute la multiplication et la longueur des chaînes.

TYPE

$\tau ::=$	num	(type des nombres)
	str	(type des chaînes)

EXPRESSION

$e ::=$	x	(variable)
	n	(nombre littéral)
	s	(chaîne littérale)
	$e_1 + e_2$	(addition)
	$e_1 * e_2$	(multiplication)
	$e_1 \,\texttt{\^{}}\, e_2$	(concaténation)
	$\texttt{len}(e)$	(longueur)
	$\texttt{let } x\texttt{=}e_1 \texttt{ in } e_2$	(définition)

Priorité :

(définition) < (concaténation) < (addition) < (multiplication)

Associativité à gauche :
addition,multiplication

Lieur :
définition : x dans e_2

Voici les règles de typage vues en CM.

$$\textsc{Num}\ \frac{}{\Gamma \vdash n : \text{num}} \qquad \textsc{Str}\ \frac{}{\Gamma \vdash s : \text{str}} \qquad \textsc{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau}$$

$$\textsc{Add}\ \frac{\Gamma \vdash e_1 : \text{num} \qquad \Gamma \vdash e_2 : \text{num}}{\Gamma \vdash e_1 + e_2 : \text{num}} \qquad \textsc{Conc}\ \frac{\Gamma \vdash e_1 : \text{str} \qquad \Gamma \vdash e_2 : \text{str}}{\Gamma \vdash e_1 \,\texttt{\^{}}\, e_2 : \text{str}}$$

$$\textsc{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \texttt{let } x\texttt{=}e_1 \texttt{ in } e_2 : \tau_2}$$

Question 1 – Donner les règles de typage pour la multiplication et la longueur.

Question 2 – Les expressions suivantes sont-elles typables ?

1. $\texttt{let } x\texttt{="}coucou\texttt{" in } x \,\texttt{\^{}}\, x$
2. $\texttt{let } x\texttt{=}x+10 \texttt{ in } x$

Question 3 – Donner un environnement dans lequel $\texttt{let } x\texttt{=}y \,\texttt{\^{}}\, y \texttt{ in len}(x)\texttt{+}z$ est typable.

Question 4 (substitution) – Prouver le lemme de substitution : pour tout Γ, x, τ_1, e_2, τ_2 tel que $\Gamma, x : \tau_1 \vdash e_2 : \tau_2$, pour tout e_1 tel que $\Gamma \vdash e_1 : \tau_1$, on a $\Gamma \vdash e_2\{e_1/x\} : \tau_2$.

On pourra ne pas faire tous les cas de la preuve par induction, mais faire au moins la variable, un autre cas de base, un lieur, et un autre opérateur.

Question 5 (décomposition) – Réciproquement, on peut toujours remplacer une sous-expression par une variable de même type dans une expression bien typée plus grande. Formaliser cet énoncé.

2 Fonctions d'ordre supérieur

On rappelle le langage avec fonctions d'ordre supérieur utilisé en CM.

TYPE

$\tau ::=$	num	(type des nombres)
	$\tau_1 \to \tau_2$	(type des fonctions)

EXPRESSION

$e ::=$	x	(variable)
	n	(nombre littéral)
	$e_1 + e_2$	(addition)
	$e_1\ e_2$	(application)
	$\texttt{fun}\ (x{:}\tau) \Rightarrow e$	(abstraction)
	$\texttt{let}\ x{=}e_1\ \texttt{in}\ e_2$	(définition)

Priorité :

(définition) < (addition)
< (application)
(abstraction) < (addition)
< (application)

Associativité à gauche :
addition,application

Associativité à droite :
type des fonctions

Lieur :

définition : x dans e_2
abstraction : x dans e

On rappelle les règles de typage spécifiques aux fonctions.

FUN
$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \texttt{fun}\ (x{:}\tau_1) \Rightarrow e : \tau_1 \to \tau_2}$$

APP
$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2}$$

Question 6 – Les expressions suivantes sont-elles typables ?
- $\texttt{fun}\ (f{:}\text{num}\to\text{num}) \Rightarrow \texttt{fun}\ (g{:}\text{num}\to\text{num}) \Rightarrow \texttt{fun}\ (x{:}\text{num}) \Rightarrow f\ (g\ x)$
- $f\ (g\ 1)\ (g\ (\texttt{fun}\ (x{:}\text{num}) \Rightarrow x))$
- $x\ x$

Question 7 – Prouver l'unicité du type résultat : pour tout e et Γ tel que $dom(\Gamma) \supseteq \text{fv}(e)$, il existe au plus un type τ tel que $\Gamma \vdash e : \tau$. Dans la preuve par induction, traiter au moins le cas de la variable, d'un lieur et d'un autre opérateur.

Question 8 (bonus) – Prouver les 3 autres lemmes (affaibliseement, substitution, décomposition).